



Application Logging using Fluentd Elasticsearch, Kibana





FluentD

+

(FEK)



Elasticsearch

+



Kibana

Logging

Logging is the most crucial aspect of any application. Every application has different flavors of the logging mechanism. However, a well-designed logging mechanism is a huge utility for the system administrators, developers. Logs are crucial to debug causes that are affecting your service.

Now as we know the value of logging in application design we will implement logging in NodeJS application using the Bunyan library. Bunyan is a simple and fast JSON logging library for NodeJS services.

Logging as application design should also be designed with questions like what, when and how much to log, how to control logging.

For eg: you cannot log a password or any secrets just for the case of debugging that's a poor way to use logging.

Logging has levels like fatal, error, info, debug, warn, trace. Every programming has it's own levels but this is the standard level available in majorly all of the logging libraries.

Fluentd

Fluentd is an open-source data collector, which lets you unify the data collection and consumption for better use and understanding of data. Fluentd is a widely used tool written in Ruby

for collecting and streaming logs to third party services like loggly, kibana, mongo for further processing.

Features

Fluentd provides tons of features we will discuss some of them.

Features

Unified Logging

Logs in JSON format are always preferable to any Logging tool. Fluentd tried to structure data as JSON as much as possible: this allows Fluentd to unify all aspects of processing log data:

collecting, filtering, buffering, and sending logs to multiple sources and destinations.

Pluggable Architecture

Fluentd has a flexible plugin system that allows the users to extend its core functionality. Users can write their own custom plugins provided they should be written in ruby.

Some of the fluentd plugins are `fluentd-elasticsearch` `fluentd-mongo` `fluentd-splunk-hec` `fluentd-kafka`.

Features

Minimum Resources

As it is written in a combination of both C as well as Ruby and requires very little system resources. Fluentd vanilla instance runs on 30–40MB of memory and can process 13,000 events/ps.

Built-in Reliability

Fluentd supports memory and file-based buffering to prevent inter-data node loss. Fluentd also supports robust failover and can be set up for high availability.

Features

Elasticsearch

Elasticsearch is a text-based search engine based on Lucene library. Elasticsearch is a database that stores, retrieves and manages document-oriented and semi-structured data. Elasticsearch relies on flexible data models to build and update visitor profiles to meet the demanding workloads and low latency required for real-time engagement.

Elasticsearch is a document-oriented database designed to store, retrieve, and manage document-oriented or semi-structured data.

Elasticsearch stores data in JSON document format just like MongoDB. Elasticsearch is also treated as schema-less unless you provide some mapping as per the application requirements. Elasticsearch features are all exposed as REST API and using Postman you can retrieve records in index, delete the index and so on. For Elasticsearch Search API it has its own DSL query language specification to search for records.

Features

Kibana

Kibana belongs to the same family as Elasticsearch both products go hand in hand and are widely used as a stack for implementing Logging as well as report generation, analytics, monitoring and so on. Kibana lets you visualize your Elasticsearch data and navigate the Elastic Stack. As data speak more when it is been visualized Kibana exactly does that by providing pools of tools for visualizing your Elasticsearch data. Kibana makes it easy to understand large volumes of data. It's simple, browser-based interface enables you to quickly create and share dynamic dashboards that display changes to Elasticsearch queries in real-time.

Features

Setting up Fluentd Linux (td-agent)

Setting up Fluentd is quite a simple and straightforward process. Before installation, we will just see what is the difference between Fluentd and td-agent.

Fluentd is an open-source data collector for a unified data logging layer. Fluentd is a project made and sponsored by Treasure Data. Treasure Data is responsible for distributing the stable version of Fluentd and which is called td-agent. So basically it's just a name difference and all the inner pieces are still of Fluentd. As we are setting up Fluentd on our own system we will not bother about pre-installation steps like ntpd and all. Just make sure you increase the max file descriptors on your system using ulimit.

Steps:

- Install from APT repository

```
curl -L https://toolbelt.treasuredata.com/sh/install-ubuntu-bionic-td-agent3.sh | sh
```

- Launch the daemon using systemd.

```
sudo systemctl start td-agent
```

- Check the status of service using below command

```
systemctl status td-agent
```

Steps:

td-agent running status

```
Activities X Terminal-emulator + Pin 12225
root@SAG-6PDJMQ2-U -
kyo@eur.ad.sag@SAG-6PDJMQ2-U - /Downloads/5woog)
root@SAG-6PDJMQ2-U - 21f5f1
root@SAG-6PDJMQ2-U: /home/kyo@eur.ad.sag/bullitio/projects/lrepo/Integration-ul# cd
root@SAG-6PDJMQ2-U:~# systemctl status td-agent
td-agent.service - td-agent: fluentd based data collector for Treasure Data
Loaded: loaded (/lib/systemd/system/td-agent.service; disabled; vendor preset: enabled)
Group: lni /etc/systemd/system/td-agent.service.d
       └─override.conf
       Active: active (running) since Thu 2019-11-14 16:28:43 IST; 19h ago
       Docs: https://docs.treasuredata.com/articles/td-agent
       Main PID: 2902 (ruby)
       Tasks: 9 (limit: 4915)
       CGroup: /system.slice/td-agent.service
               └─2902 /opt/td-agent/embedded/bin/ruby /opt/td-agent/embedded/lib/ruby/gems/2.4.0/gems/fluentd-1.7.0/lib/fluent/command/fluentd.rb --log /var/log/td-agent/td-agent.log --daemon /var/run/td-agent/td-ag
Nov 14 16:28:43 SAG-6PDJMQ2-U fluentd[2862]: Using system 2.5.2
Nov 14 16:28:43 SAG-6PDJMQ2-U fluentd[2862]: Using ohai 6.20.0
Nov 14 16:28:43 SAG-6PDJMQ2-U fluentd[2862]: Using fluent-plugin-tid-monitoring 0.2.3
Nov 14 16:28:43 SAG-6PDJMQ2-U fluentd[2862]: Using webhdfs 0.6.0
Nov 14 16:28:43 SAG-6PDJMQ2-U fluentd[2862]: Using 'fluent-plugin-webhdfs' 1.2.2
Nov 14 16:28:43 SAG-6PDJMQ2-U fluentd[2862]: Bundle complete! 13 Gemfile dependencies, 58 gems now installed.
Nov 14 16:28:43 SAG-6PDJMQ2-U fluentd[2862]: Bundled gems are installed into `./etc/td-agent/vendor/bundle`
Nov 14 16:28:43 SAG-6PDJMQ2-U fluentd[2862]: No entry for terminal type "dumb";
Nov 14 16:28:43 SAG-6PDJMQ2-U fluentd[2862]: using dumb terminal settings.
Nov 14 16:28:43 SAG-6PDJMQ2-U systemd[1]: Started td-agent: Fluentd based data collector for Treasure Data.
root@SAG-6PDJMQ2-U:~#
```

Steps:

Init.d

After installation of td-agent you will be provided with startup scripts (/etc/init.d/td-agent) to manage td-agent daemon.

Setting up Elasticsearch

Before installing ELK make sure you have Java 8 if not you can install it using `sudo apt install openjdk-8-jdk`.

Setting up Kibana using Docker

Kibana and Elasticsearch release are linked with each other so for every release of ELK a same version of kibana will be published. Every version of kibana is only compatible with the same version of ELK.

For avoiding a mess on our system if things go terribly wrong we will use the docker image of kibana at Docker Image Kibana. As I have ELK 7.1.1 I will pull Kibana with tag 7.1.1 using `docker pull`.

Steps:

To run kibana use below command

```
docker run -d --name kibana --net somenetwork -p 5601:5601  
kibana:7.1.1
```

You can skip specifying network unless you are linking two containers.

If you want to use Elasticsearch running on your host machine use below command to connect to your local Elasticsearch instance:

```
docker run -d --name kibana --net host -e  
"ELASTICSEARCH_HOSTS=http://localhost:9200" kibana:7.1.1
```

Steps:

You can verify kibana container running and connecting to your Elasticsearch using below command:

NOTE: By default, kibana image assumes ELK container with “elasticsearch” name exists within the same network and so tries to connect using `http://elasticsearch:9200` using the service discovery feature of docker.

NodeJS API Server



```
1  const bunyan      = require('bunyan');
2  const format      = require('bunyan-format');
3  const express     = require('express');
4  const bodyParser  = require('body-parser');
5  const FS          = require('fs');
6  const _           = require('lodash');
7
8  global.logger = configureLogger();
9
10 const app = express();
11
12 // for logging request
13 const logRequest = function (req, res, next) {
14   const start = new Date();
15   const end   = res.end;
16
17   res.end = function(payload, encoding){
18     let responseTime = Date.now() - start.getTime();
19     end.call(res, payload, encoding);
20     const contentLength = Number(res.getHeader('Content-Length'));
21     const _data = {
22       req: _.pick(req, ['method', 'url']),
23       res: _.pick(res, ['statusCode']),
24       responseTime: responseTime,
```

NodeJS API Server

```
24     responseTime: responseTime,  
25     contentLength: isNaN(contentLength) ? 0 : contentLength  
26   }  
27   logger.info('%s %s %s %d %dms - %d bytes', (req.hostname || ''), _data.req.method, _data.req  
28   )  
29   next();  
30 }  
31  
32 app.use(logRequest());  
33 app.use(bodyParser.json());  
34  
35 function configureLogger(){  
36   const logFile = './api.LOG';  
37   const ws = FS.createWriteStream(logFile, { flags: 'a' });  
38  
39   // formatting options for stdout  
40   const formatOut = format({ outputMode: 'short', levelInString: true, colorFromLevel: {  
41     20: 'blue',  
42     30: 'green',  
43     40: 'yellow',  
44     50: 'red',  
45     60: 'brightRed'  
46   }});  
47  
48  
49   const fileFormatter = format({outputMode: 'bunyan', level: true }, ws);  
50  
51   const logger = bunyan.createLogger({  
52     name: 'API Logger',
```

NodeJS API Server

```
53   level: 'debug',
54   streams: [
55     {
56       level: 'debug',
57       stream: fileFormatter
58     },
59     {
60       level: 'info',
61       stream: fileFormatter
62     },
63     {
64       level: 'debug',
65       stream: process.stdout
66     }
67   ]
68 });
69
70 logger.log = function (message) {
71   logger.debug(message);
72 }
73 return logger;
74 }
75
76 app.listen(4044, () => {
77   logger.log("API Server Listening on PORT 4044");
78 });
```


Configure FluentD

Configuring Fluentd or td-agent is straight forward and all the plugins configuration are written in fluentd.conf or td-agent.conf. Config location can be either :

```
/etc/td-agent/td-agent.conf
```

```
// or
```

```
/etc/fluentd/fluent.conf
```

As we are using Elastic search plugin for sending data out to Elasticsearch we will need fluentd-plugin-elasticsearch output plugin which you can install using gem.

```
gem install fluent-plugin-elasticsearch 2.4.0
```

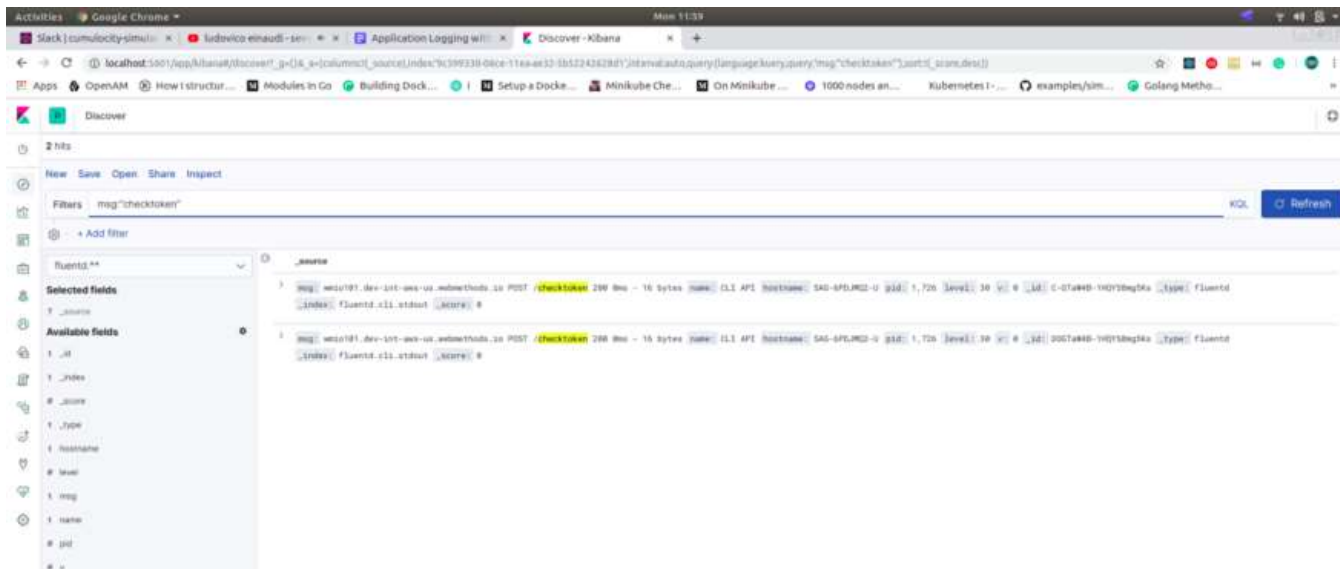
Configure FluentD

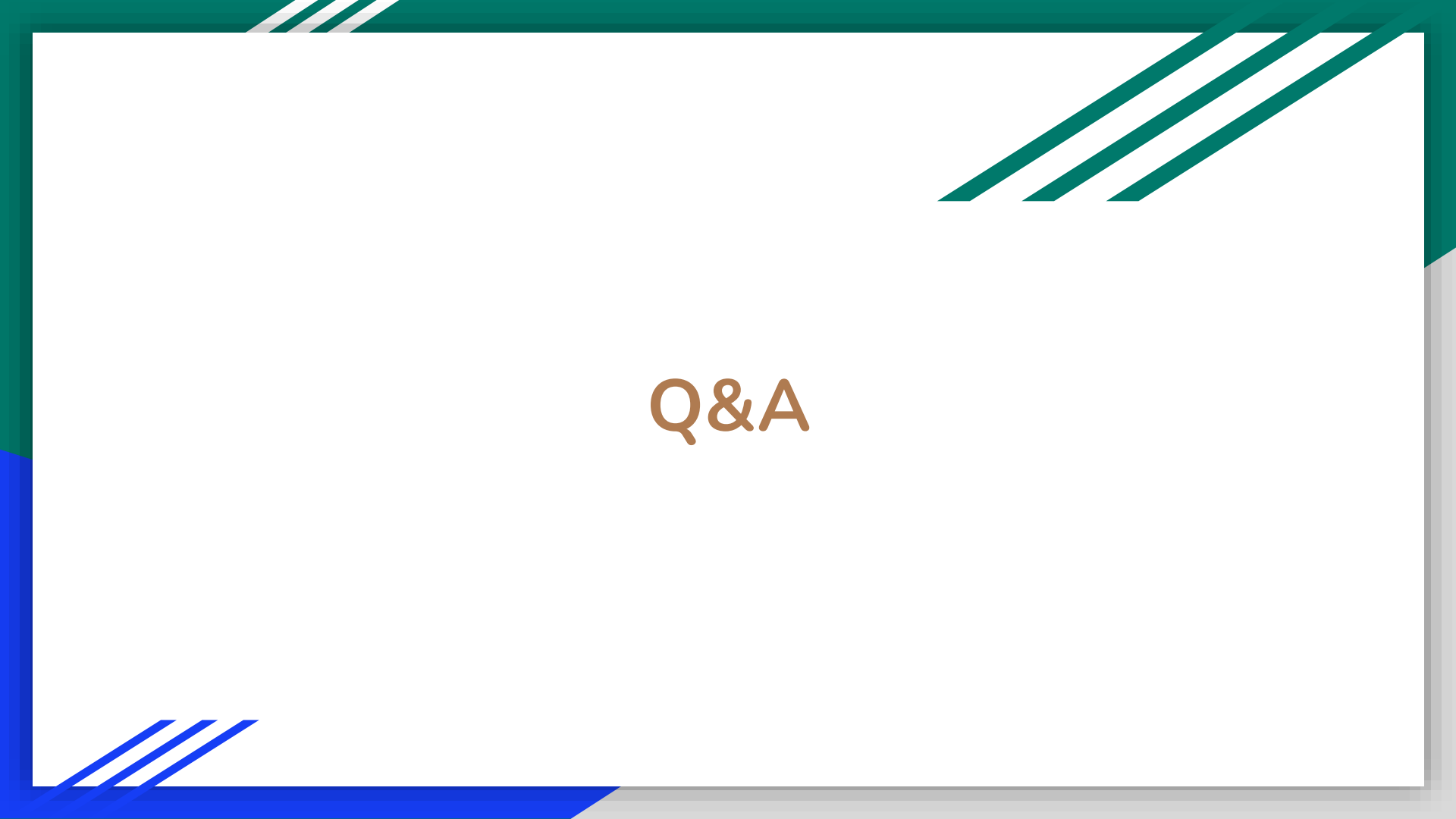
```
1 <system>
2   workers 1
3   log_level trace
4 </system>
5
6 # logs for api
7 <source>
8   @type tail
9   format json
10  tag cli.stdout
11  path /home/kyo@eur.ad.sag/builtio/projects/irepo/integration-cli/logs/cli.LOG
12  pos_file /var/log/td-agent/tmp/cli.log.pos
13 </source>
14
15 # send logs of application to kibana via elastic search
16 <match cli.stdout>
17   @type elasticsearch
18   host localhost
19   port 9200
20   index_name fluentd.${tag}
21   flush_interval 5s
22 </match>
```

Searching logs in Kibana

Once the plugin configuration is setuped you can reload fluentd to reflect new plugins configuration. You can open kibana dashboard on your machine at localhost:5601

And search for messages in the search box using DSL.





Q&A